

Lab 4 实验报告

实验目的

1. 掌握 CPU 的外设 I/O 模块的设计方法。理解 I/O 地址空间的译码设计方法；
2. 掌握 Vivado 仿真、实现、板级验证方式；
3. 通过扩展新指令的实现，深入理解 CPU 对指令的译码、执行原理和实现方式。

实验平台及器材

1. 计算机 1 台 (尽可能达到 8G 及以上内存) ；
2. Xilinx 的 Vivado 开发套件 (2020.2 版本) ；
3. Xilinx 的 EGO1 FPGA 开发板。

实验任务

概述

本次实验的主要任务是在实验三完成的单周期 CPU 核基础上进行外设拓展，增加 CPU 对 I/O 口的读写支持，以及对新指令的功能支持，并在 EGO1 实验板上完成板级测试验证。

文件树展示



图1: 文件树

I/O模块设计原理

I/O 空间和数据存储器空间是统一编址的，用高端地址空间作为 IO 空间，并用高位地址线进行译码。以 `io_input` 模块为例：

```
case (sel_addr)
    6'b100000: y = a0; // inport0 byte address 0x80
    6'b100001: y = a1; // inport1 byte address 0x84
    default: y = 32'h0;
endcase
```

这段二位选择器代码决定了输入端口地址（0x80、0x84）。

I/O模块代码修改

按照图2给出的框架图，在Lab 3的实验代码基础上修改、增添，逐步完成有关模块的链接。

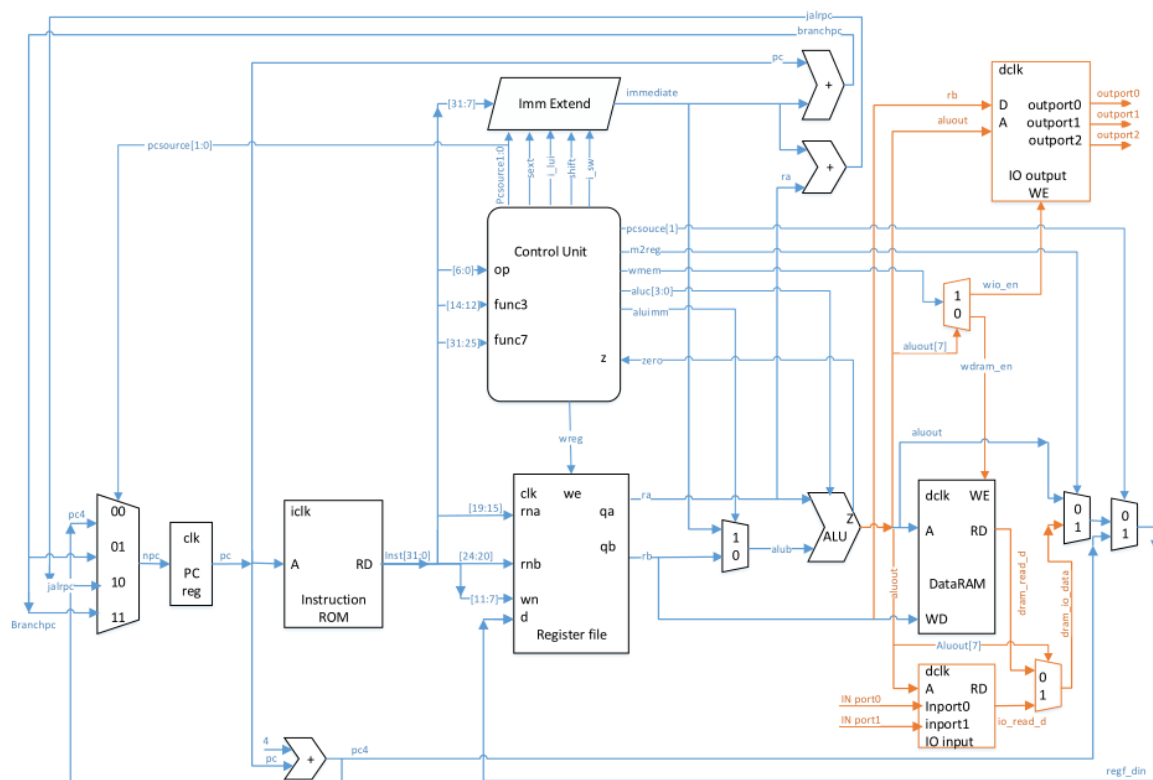


图2：支持22条指令的单周期 CPU 扩展 I/O 框架图

1. 在 `sc_datamem` 模块中完成子模块 `io_input` 和 `io_output` 的例化：

```
//IO output , io_output ,add here
io_output io_output_reg (
    .out_port0(out_port0),
    .out_port1(out_port1),
    .out_port2(out_port2),
    .addr(addr),
    .datain(datain),
    .write_io_enable(write_io_enable),
    .io_clk(dmem_clock),
    .resetn(resetn)
);

//IOinput , io_input ,add here
io_input io_input_reg(
    .addr(addr),
    .io_clk(dmem_clock),
    .io_read_data(io_read_data),
    .in_port0(in_port0),
    .in_port1(in_port1)
);
```

2. 在软核顶层 `sc_computer_main` 模块中例化 `sc_instmem` 和 `sc_datamem` 两个存储器模块:

```
//sc_instmem instruction memory.
sc_instmem imem(
    .addr(pc),
    .inst(inst),
    .clock(clock),
    .imem_clock(imem_clock)
);

//sc_datamem data memory and IO module.
sc_datamem dmem (
    .resetn(resetn),
    .addr(aluout),
    .datain(data),
    .we(wmem),
    .dataout(memout),
    .clock(clock),
    .dmem_clock(dmem_clock),
    .out_port0(out_port0),
    .out_port1(out_port1),
    .out_port2(out_port2),
    .in_port0(in_port0),
    .in_port1(in_port1)
);
```

注意到本实验使用了一个独立的时钟生成模块 `clock_and_mem_clock` 为指令存储器和数据存储器分别生成 `dmem_clock` 和 `imem_clock`，在例化时要小心传参，避免时钟混用导致错误。

3. 在整个项目的顶层文件 `sc_cpu_iotest` 中例化 `sc_computer_main` 等下层模块

```
//sc_computer_main unit
sc_computer_main computer_main(
    .resetn(sys_rst_n), //系统时钟
    .clock(clock_out),
    .imem_clock(imem_clock),
    .dmem_clock(dmem_clock),
    .in_port0(inport0), // input data1
    .in_port1(inport1), // input data2
    .out_port0(out_port0), // output data1
    .out_port1(out_port1), // output data2
    .out_port2(out_port2) // output data3
);
```

本实验模块层级较多，且部分变量会反复作为输入/输出被传递，例化时应特别注意不同文件中定义的变量名可能不同，要准确对应。

扩展指令原理

先要求扩展 ALU 模块功能，使其支持求两个 32 比特数的汉明距离的操作。为此，我们自定义该操作为 R-type 指令 `hamd rd, rs1, rs2`，其编码中 `func3=111`，`func7=0100000`，`op=0110011`，则 `hamd x16, x14, x15` 的指令码为 `0100000|01111|01110|111|10000|0110011 = 40f77833`。

指令扩展代码修改

1. `sc_cu` 模块中为新指令（`hamd`）定义控制变量 `i_hamd`，并为其分配不相冲突的 `aluc` 代码 1111：

```
wire i_hamd = r_type & (func3 == 3'b111) & (op == 7'b0110011);
assign aluc[3] = i_sub | i_sra | i_srai | i_beq | i_bne | i_hamd; //增加i_hamd
assign aluc[2] = i_and | i_or | i_xor | i_srl | i_sra | i_hamd | //增加i_hamd
               i_andi | i_ori | i_xori | i_srli | i_srai ;
assign aluc[1] = i_and | i_or | i_andi | i_ori | i_lui | i_hamd; //增加i_hamd
assign aluc[0] = i_and | i_sll | i_srl | i_sra | i_andi | i_hamd | //增加i_hamd
               i_slli | i_srli | i_srai ;
assign wreg = i_add | i_sub | i_and | i_or | i_xor | i_hamd | //增加i_hamd
             i_sll | i_srl | i_sra | i_addi | i_andi |
             i_ori | i_xori | i_srli | i_slli | i_srai | i_lw | i_jalr | i_lui |
             i_jal;
```

2. `alu` 模块增添汉明距离运算操作（其他运算代码略去）

```
integer i,cnt ;
reg [31:0] s;
reg      z;
always @ (a or b or aluc)
begin
    ////////// 以下开始求汉明距离 //////////
    if(aluc == 4'b1111) begin //判定aluc == 1111
        cnt = 0; //计数相异的位数
        for(i=0; i<31; i=i+1)begin
            if(a[i]!=b[i]) cnt = cnt + 1; //循环比较每一位，相异则计数+1
        end
        s = cnt; // 将计数赋值给aluc输出
    end
    ////////// 结束求汉明距离 //////////
end
```

这里应当注意：使用循环语句应使用 `always` 过程赋值，且变量类型相应更改为 `reg` 类型（区别于此前 `assign` 语句对 `wire` 变量赋值）。

仿真结果展示

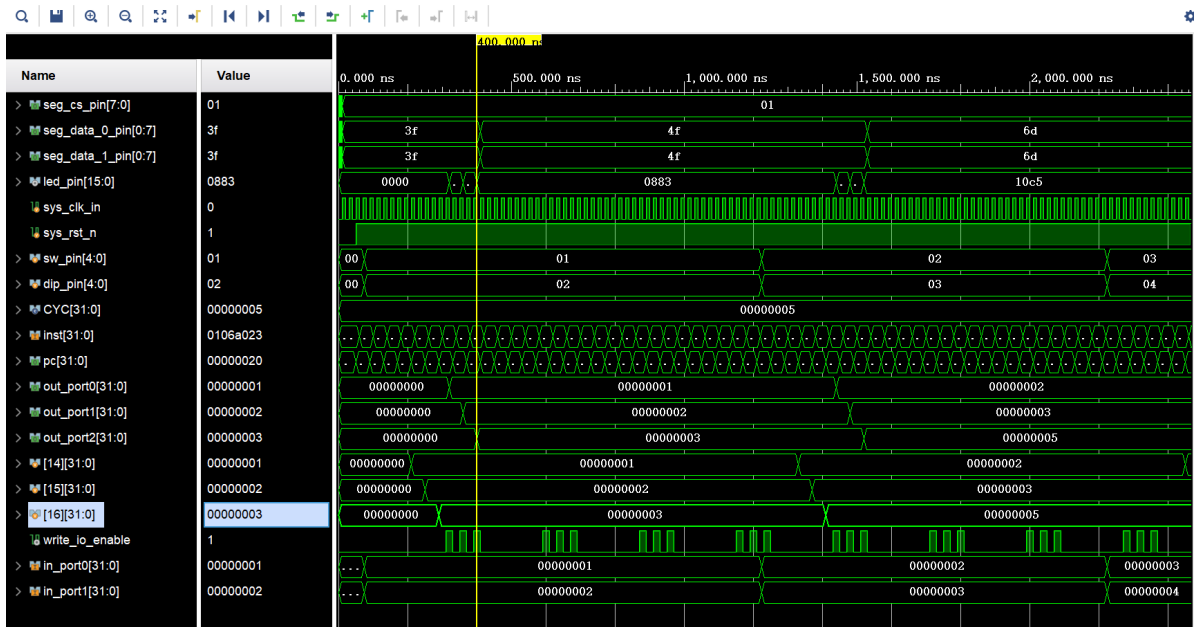


图3: I/O扩展仿真

从图3中可以看出，400 ns时，正确完成第一轮循环，out_port0 和 out_port1 正确显示了 in_port0 和 in_port1 的数据，并在 out_port2 中正确得到了二者加和的结果（ $1+2=3$ ， $2+3=5$ ，此后同理）。

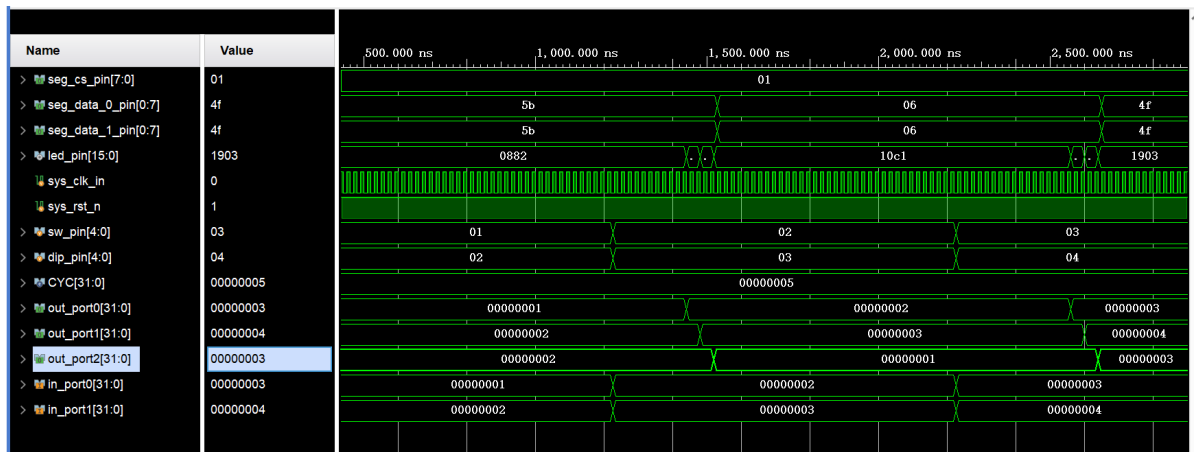


图4: 扩展指令（汉明距离）仿真

从图4中可以看出，程序正确完成循环，out_port0 和 out_port1 正确显示了 in_port0 和 in_port1 的数据，并在 out_port2 中正确得到了二者求汉明距离的结果（ $hamd(1,2)=2$ ， $hamd(2,3)=1$ ，此后同理）。

板上验证结果演示

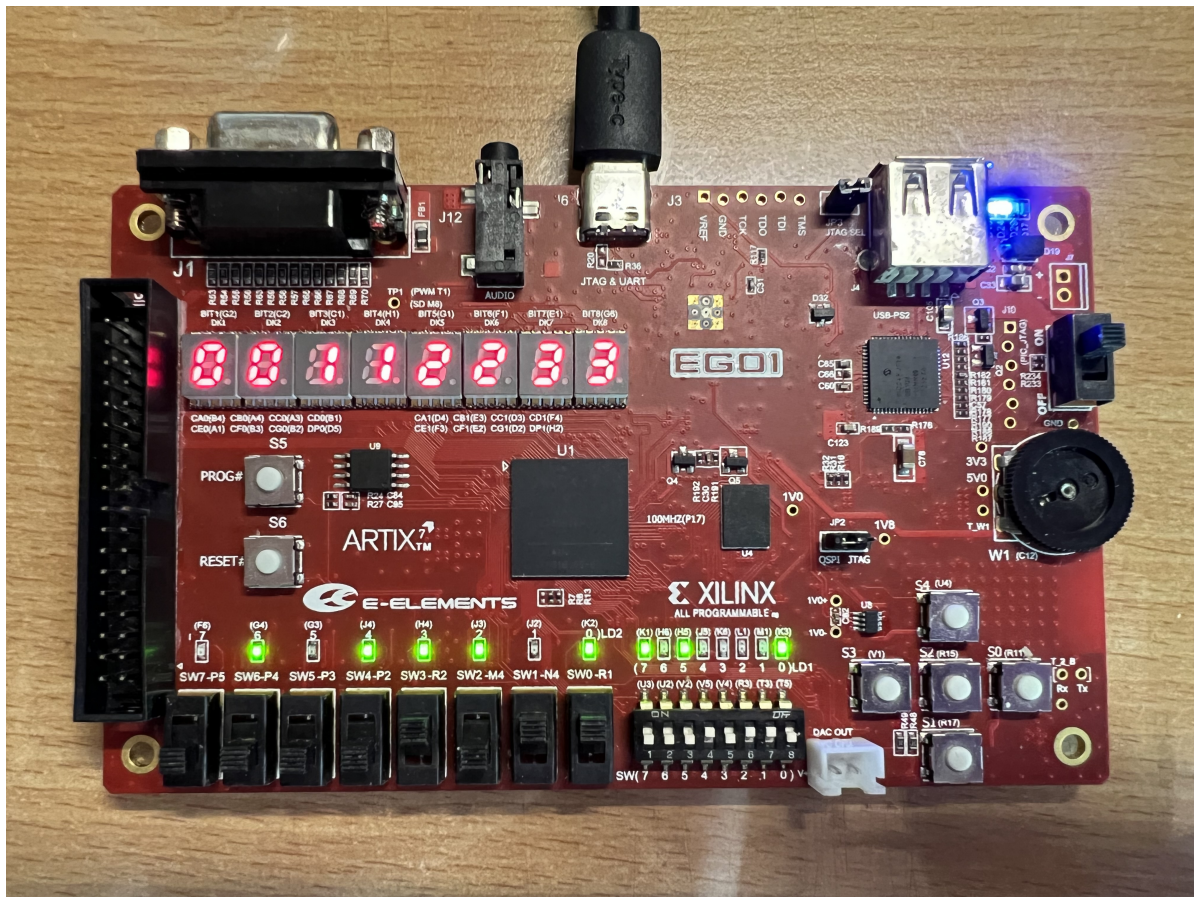


图5: I/O支持板上验证

如图5所示，通过FPGA板上开关分别输入二进制数 `01011` 和 `10110`，在LED数码管显示区正确显示（十进制）两输入数 `11`、`22` 及其求和结果 `33`。

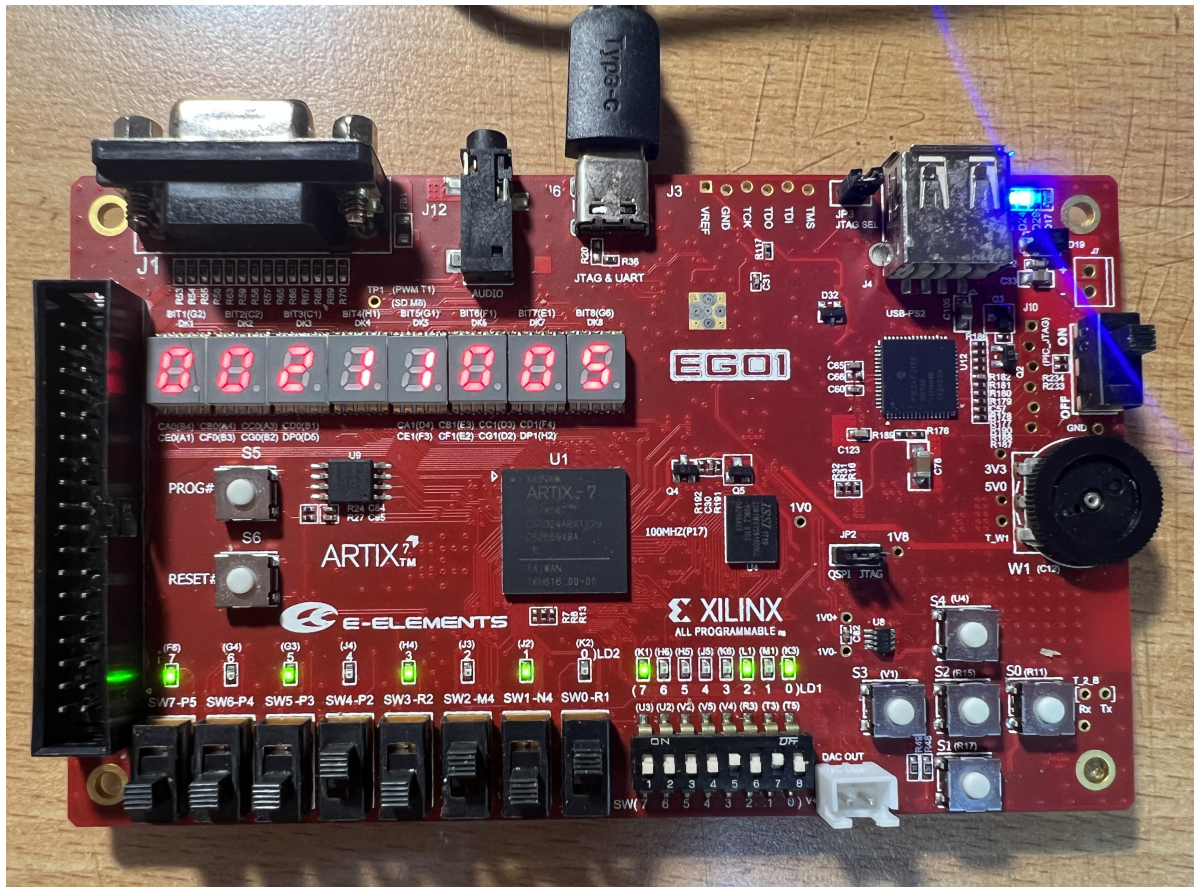


图6: 扩展指令 (汉明距离) 板上验证

如图6所示, 通过FPGA板上开关分别输入二进制数 `10101` 和 `01010`, 在LED数码管显示区正确显示 (十进制) 两输入数 `21`、`10` 及其汉明距离 `5`。

(解释: `10101` 与 `01010` 互为取反, 异或和为 `5`, 故汉明距离为 `5`)

总结

- 通过仿真验证和板级验证, 本次实验的两大核心任务 (I/O扩展、指令扩展) 均正确完成并得到有效验证, 对有关数据、指令、译码等CPU工作细节有了进一步深入的理解和掌握。
- 值得指出, 我在实验过程中遇到了一些问题, 在解决它们的过程中对vivado平台的verilog编程及仿真实践有了很大的提高。如先后出现将变量名对应错误 (`.reset_n(sys_rst_n)` 打成了 `.reset_n(resetn)`) 导致取指异常; 将时钟信号对应错误 (`clock` 和 `clock_out` 混用) 等小错误, 通过仿真结果可以逐步倒推检查错误位置并予以修正, 该过程获益匪浅。